

Documentación Técnica de Base de Datos

Diseño Físico y Modelo Relacional de Datos

Diseño y Administración de Base de Datos: *Isaac Arias*

Entorno de Desarrollo: *Devbox Shell (PostgreSQL 18)*

Entorno de Producción: *AWS RDS PostgreSQL 18*

Fecha: *Julio de 2026*

Índice

Resumen Ejecutivo.....	3
MÓDULO 1: NÚCLEO TRANSACCIONAL E INTELIGENCIA ARTIFICIAL (RAG).....	4
1. Tabla: Documento.....	4
2. Tabla: DocumentoChunk.....	5
MÓDULO 2: MOTOR DE TRAZABILIDAD Y LINAJE JURÍDICO REFLEXIVO.....	7
3. Tabla: RelacionDocumental.....	7
4. Tabla: TipoRelacion.....	9
5. Tabla: RespaldoLegal.....	9
MÓDULO 3: TAXONOMÍAS Y ESTRUCTURAS JERÁRQUICAS (PADRE-HIJO).....	10
6. Relación: TipoPrograma → NivelAcademico → Carrera.....	10
7. Relación: MacroCategoria → Categoria.....	10
8. Relación: TipoArea → subAreas.....	10
MÓDULO 4: DICCIONARIOS DE CONFIGURACIÓN Y CONTEXTO (LOOKUPS).....	11
MÓDULO 5: MATRIZ DE RELACIONES MULTIVALOR (TABLAS PUENTE N:M).....	12
Notas de Integridad de Datos para el Administrador:.....	15
Script de creación de las tablas: (ACTUALIZADO 16/jul 23:02).....	16
MÓDULO 6: Extensiones.....	31
1. Introducción y Contexto.....	31
2. Devbox.....	31
3. Arquitectura del Entorno y Paquetes Seleccionados.....	32
4. Proceso de Instalación y Configuración.....	33
Fase 1: Inicialización del Entorno de Desarrollo.....	33
Fase 2: Configuración del Servidor.....	33
Desarrollo:.....	33
Producción:.....	34
Fase 3: Activación en el Motor de Base de Datos.....	34
Fase 5: Instalación y configuración de pg_permissions.....	34
Comando ejecutado en la terminal de devbox shell:.....	34
Resultado de la Ejecución.....	35
5. Control de Calidad y Verificación.....	35
MÓDULO 7: Optimización y mejoras.....	36
1. Sincronización Nativa y Garantía del Índice Léxico en el Motor.....	36
1.1 Diagnóstico de Riesgos Operativos en la Gestión desde la Capa de Aplicación.....	36
1.2 Fundamentación Técnica de la Solución Nativa en PostgreSQL.....	37
2. Control de Acceso basado en roles.....	38
2.1 Justificación Técnica y Principio de Menor Privilegio (POLP).....	38
2.2 Matriz de Definición de Roles del Sistema.....	39
2.3. Script DDL de Fortalecimiento (Hardening) y Asignación de Permisos.....	40
Fase A: Revocación de Permisos Predeterminados (PUBLIC).....	40
Fase B: Configuración del Rol para el Backend (giru_app_user).....	40
Fase C: Configuración del Rol de Auditoría (giru_auditor_user).....	41
Fase D: Automatización de Permisos Futuros (Default Privileges).....	41

Resumen Ejecutivo

El Proyecto implementa una arquitectura de base de datos relacional sobre PostgreSQL 18, utilizando Devbox para la creación de entornos de desarrollo locales aislados y reproducibles, y AWS RDS PostgreSQL 18 como infraestructura de producción, cuya migración de estructura y datos iniciales se realizó mediante `pg_dump`. La tabla principal `Documento` actúa como la Única Fuente de Verdad (SSOT) desvinculada de los archivos pesados almacenados en AWS S3, mientras que la tabla `DocumentoChunk` sustenta el motor de Inteligencia Artificial (RAG) mediante un esquema de búsqueda híbrida. Este motor opera con la extensión `pgvector` mediante un índice HNSW para vectores de 1024 dimensiones (modelo BGE-M3), combinada nativamente con una columna generada `tsvector` en español para consultas léxicas de alta velocidad y estricta consistencia ACID transaccional.

El diseño físico satisface la Tercera Forma Normal (3FN) para optimizar el uso de memoria RAM, resolviendo relaciones multivalor complejas a través de tablas puente con llaves primarias compuestas que ejecutan intersecciones de filtros en microsegundos. La trazabilidad y el linaje normativo se estructuran en la tabla `RelacionDocumental` como un grafo acíclico dirigido (DAG), protegido por un disparador `BEFORE` con CTE recursiva que intercepta y bloquea bucles circulares infinitos antes de tocar el disco, y un disparador `AFTER` que actualiza la vigencia y desactiva (`isActive = FALSE`) los documentos derogados para prevenir alucinaciones en el bot RAG. Las políticas de integridad referencial aplican `ON DELETE CASCADE` en tablas transaccionales y `ON DELETE RESTRICT` en catálogos maestros para resguardar el registro histórico.

La estrategia de ciberseguridad se rige por un modelo RBAC bajo el Principio de Menor Privilegio (POLP), revocando permisos implícitos al pseudorol `PUBLIC` y automatizando privilegios futuros. La estructura segrega tres perfiles: `giro_app_user` para operaciones DML del backend, `giro_auditor_user` en modo lectura estricta (Read-Only) y `giroAdmin` para la gestión DDL del esquema. La auditoría y monitoreo se apoyan en extensiones como `pgaudit`, `pg_permissions`, `pg_cron` y `pg_stat_statements`, parametrizadas en producción mediante **DB Parameter Groups** en AWS RDS. Cada fragmento incorpora un hash criptográfico `hashSha256` para garantizar el no repudio e integridad de la información, en cumplimiento directo con la ISO 27001, la Ley 21.663 de Ciberseguridad en Chile y la Ley 21.719 sobre Protección de Datos Personales.

MÓDULO 1: NÚCLEO TRANSACCIONAL E INTELIGENCIA ARTIFICIAL (RAG)

1. Tabla: **Documento**

Es el eje central del ecosistema y la **Única Fuente de Verdad (SSOT)**. Almacena la metadata de identidad e integridad de cada norma, decreto o resolución de la universidad.

- **Atributos de Control:** **idDocumento** (PK) , **numero** (Código único institucional) , **titulo** , **descripcion** y los flags operativos **aplicacionInmediata** e **isActive**.
- **Segregación de Almacenamiento Estático:** Define las rutas hacia el almacenamiento en la nube S3 (**urlArchivoOriginalS3** y **urlOcrS3**), aislando los archivos pesados de la transaccionalidad de PostgreSQL.
- Metadatos de Contexto e Identidad Integrados:
 - **numSesion** (Contexto): Identifica la reunión física o virtual en el calendario del Consejo. Ejemplo: La junta celebrada el día 3 de enero de 2019 fue la Sesión Ordinaria N° 337.
 - **numAcuerdo** (Identidad/Contenido): Identifica la decisión o resolución específica aprobada dentro del acta de esa sesión. Ejemplo: En la sesión N° se aprobó modificar un reglamento bajo el Acuerdo N° 1493; la posterior creación de una carrera esa misma tarde correspondería al Acuerdo N° 1494.
 - **nomMetaDato** (Identidad): Identifica el documento por su metadato, el cual sigue el siguiente formato:

<macroCateogria>-<categoria>-numeroIdentificador(único en categoria)
Este código (llámese “metadato”) debe ser único por documento. Sin embargo no se catalogará como una primary Key, el cual permitirá una búsqueda más avanzada y con precisión evitando aprenderse la Primary key de memoria.
- **Optimización de Rendimiento e Indexación Local:**
 - ° Índice **idx_doc_numero**: Estructura de tipo **B-Tree** tradicional aplicada sobre la columna **numero**.
 - **Propósito Técnico:** Optimiza las búsquedas e integraciones con sistemas externos. Cuando un analista o el backend acota una consulta por un número oficial de decreto (ej: *"Buscar solo coincidencias en el Decreto N° 052/2008"*), este índice permite a PostgreSQL localizar el registro maestro instantáneamente $O(\log n)$, eliminando por completo los costosos escaneos secuenciales de tabla (*Sequential Scans*).

2. Tabla: DocumentoChunk

Tabla satélite de gran granularidad diseñada como el núcleo operativo para la Inteligencia Artificial, el motor RAG (Retrieval-Augmented Generation) y la citación legal de alta precisión mediante Búsqueda Híbrida.

- **Mecánica Operativa:** Divide el texto completo de las normativas universitarias en fragmentos atómicos secuenciales (`numeroPagina`, `secuencia`), vinculados a la tabla maestra mediante la llave foránea (`idDocumento`).
- **Metadatos de Estructura Jurídica (Citación de Alta Precisión):**
 - `nombreTitulo (VARCHAR(255))`: Almacena el nombre del título o sección mayor del reglamento al que pertenece el fragmento (ej: "*Título III: Del Rendimiento Académico y la Permanencia*").
 - `numeroArticulo (VARCHAR(50))`: Almacena el identificador específico del artículo (ej: "*Artículo 27*" o "*Art. Transitorio*").
 - `numeroInciso (VARCHAR(50))`: Identifica el párrafo o inciso exacto dentro del artículo (ej: "*Inciso 1*", "*Inciso segundo*").
- **Clasificación Estructural:**
 - `idTipoPagina (FK)`: Enlace referencial con la tabla `TipoPagina` que categoriza el contexto del chunk para un filtrado exacto en las consultas semánticas.
- **Motor Semántico RAG (Crítico - pgvector):** Incorpora el campo `embedding` de tipo `VECTOR(1024)` (requiere la extensión `pgvector` habilitada en la instancia). Este campo almacena la representación matemática tridimensional del texto con una **dimensión de 1024**, correspondiente al tamaño de salida nativo del modelo de embeddings **BGE-M3** seleccionado para el proyecto. Esto permite al chatbot ejecutar búsquedas semánticas y conversacionales de alto nivel (ej. *¿Cuáles son los requisitos para postular al magíster?*), comprendiendo la intención y el contexto de la consulta más allá de las palabras exactas.
- **Métrica de Distancia y Similitud:** Se asocia directamente al operador de búsqueda `vector_cosine_ops` (Similitud Coseno), debido a que el modelo BGE-M3 fue pre-entrenado bajo esta métrica específica para medir la cercanía semántica entre textos.
- **Estrategia de Indexación (HNSW):** Se implementa un índice de tipo **HNSW (Hierarchical Navigable Small World)** utilizando similitud coseno. HNSW ofrece una calidad y precisión de búsqueda muy superior a *IVFFlat* y no requiere fases de entrenamiento previo. Para el volumen proyectado de la institución (estimado entre **10,000 y 20,000 chunks**), el costo de memoria RAM adicional requerido por HNSW es totalmente despreciable.
- **Política de Nullability (Flexibilidad del Pipeline):** La restricción `NOT NULL` se define como **opcional (campo nullable)**. Esto otorga un desacoplamiento operativo crucial en el backend: permite al pipeline de ingesta registrar los fragmentos de texto rápidamente en la base de datos y calcular/actualizar los embeddings de forma asíncrona mediante un proceso en segundo plano (*worker/batch pipeline*), evitando bloquear la transacción principal del usuario.

- **Búsqueda Léxica Integrada (Búsqueda Híbrida):** Almacena el tipo de dato nativo `TSVector` (`indiceLexico`) para ejecutar búsquedas tradicionales de palabras clave. Al combinarse con el campo `embedding`, PostgreSQL puede realizar pre-filtrados instantáneos y ordenaciones por similitud híbrida.
- **Impacto en Espacio en Disco:** Cada embedding de 1024 dimensiones en formato *Float32* consume exactamente **4 KB** de espacio físico en disco. Para el universo máximo proyectado de 20,000 chunks, el almacenamiento incremental es de apenas **~80 MB extras**, una cifra irrelevante para los servidores de producción de PostgreSQL.
- **Seguridad e Integridad:** Emplea un `hashSha256` por fragmento (de longitud exacta de 64 caracteres) que garantiza criptográficamente que ningún bloque de texto ha sido alterado, cumpliendo con las directrices de integridad de la norma **ISO 27001** y la **Ley 21.663** de Ciberseguridad en Chile.

MÓDULO 2: MOTOR DE TRAZABILIDAD Y LINAJE JURÍDICO REFLEXIVO

3. Tabla: RelacionDocumental

Tabla puente de autorreferencia (vínculo reflexivo de Muchos a Muchos) encargada de mapear el linaje, la trazabilidad jurídica y la evolución histórica de cómo las normativas universitarias interactúan, se modifican o se derogan entre sí a lo largo del tiempo.

- **Mecánica Relacional:** Conecta un documento emisor (`idDocumentoOrigen`) con el documento que recibe el impacto legal (`idDocumentoDestino`), amarrados bajo un tipo de relación específico (`idTipoRelacion`) y respaldados por un decreto o acta (`idRespaldoLegal`).
- **Atributo Clave de Contenido:**
 - `detalleModificacion` (`TEXT`): Almacena el impacto específico y único de la interacción jurídica (ej: *"Modifica el Artículo 27"*, *"Deroga el Título III completo"*). Permite a la Secretaría Académica conocer el detalle milimétrico de la afectación sin necesidad de abrir y leer manualmente el PDF completo.
- **Trazabilidad de Detección y Confiabilidad (Soporte AI/RAG):**
 - `idFuenteDeteccion` (`FK`): Enlace referencial con la tabla `FuenteDeteccion` que identifica cómo se descubrió este vínculo en el pipeline (*Ingreso Manual, Regex, Nombre de Archivo*).
 - `confianza` (`NUMERIC(3,2)`): Almacena un valor decimal de certeza (entre `0.00` y `1.00`). Un valor de `1.00` indica certeza total (ej. ingreso manual o validación humana), mientras que valores menores (ej. `0.85` por detección automática regex) indican que la relación requiere una auditoría visual antes de ser procesada por el sistema.
 - `verificada` (`BOOLEAN`): Flag de control administrativo. Si es `FALSE`, la relación queda en cola de revisión para el equipo de gestión documental; si es `TRUE`, significa que ha sido confirmada por un humano y está activa para el negocio. Para inserciones manuales tradicionales o herencia histórica, el valor por defecto es `TRUE` con el fin de asegurar compatibilidad hacia atrás.
 - `textoEvidencia` (`TEXT`): Fragmento o párrafo exacto del OCR donde el modelo automático o regex identificó la derogación/modificación. Funciona como bitácora y soporte visual directo para el proceso de auditoría humana.
- **Seguridad de Datos y Prevención de Duplicados (Índice Único de Negocio):**
 - Se implementa una restricción de unicidad estricta mediante el índice `idx_rel_unica` sobre la tupla compuesta por (`idDocumentoOrigen`, `idDocumentoDestino`, `idTipoRelacion`). Esta regla blindará al sistema contra corrupciones de datos provocadas por la redundancia (por ejemplo, que el parser automático inserte una derogación que un analista ya había ingresado a mano).

- **Cómo funciona en el motor:** Si alguien o algún script automatizado intenta insertar una relación redundante que ya existe con la misma combinación exacta de origen, destino y tipo, PostgreSQL abortará la transacción de forma segura y rechazará el **INSERT** arrojando el siguiente error nativo: **ERROR: duplicate key value violates unique constraint "idx_rel_unica"** El backend de la aplicación puede capturar esta excepción estructurada de manera limpia y transformarla en un mensaje de cara al usuario del tipo: *"Esta relación ya existe en el sistema"*.
- **Justificación de los campos elegidos:** La combinación de estos tres campos representa la **identidad lógica** de una interacción institucional. Carece de sentido de negocio que coexistan dos registros que indiquen que el "Documento A deroga al Documento B". Sin embargo, el diseño permite flexibilidad para combinaciones cruzadas válidas: el Documento A *SÍ* puede derogar y modificar simultáneamente al Documento B (serían dos relaciones distintas con tipos diferentes), o el Documento A puede derogar al B mientras el B modifica al Documento C (contextos lógicos independientes).
- **Protección Avanzada del Grafo (Prevención de Ciclos Indirectos / Transitivos):**
 - Con el fin de evitar bucles circulares que corrompan lógicamente la jerarquía y confundan al chatbot (ej: *Documento A modifica a B, B modifica a C, y C modifica a A*), la tabla incorpora una validación por **Árbol de Recursividad** en caliente a través de un trigger procedimental **BEFORE**.
 - **Mecánica del Algoritmo:** Cada vez que se intenta registrar o actualizar una relación, el motor ejecuta una Expresión de Tabla Común (CTE) recursiva (**WITH RECURSIVE**) navegando por todos los caminos del grafo normativo río abajo. Si el destino propuesto vuelve a conectar de forma directa o indirecta con el origen de la operación, el motor detecta el bucle infinito, aborta el almacenamiento físico y devuelve una excepción controlada: **ERROR: Violación de Grafo Acíclico (DAG): La relación propuesta entre el documento origen ... genera un bucle circular infinito.**
- **Automatización Operativa (Trigger de Sincronización de Vigencia Semántica):**
 - Para erradicar el "error silencioso" de mantener normativas obsoletas como vigentes en la base de datos (lo que provocaría alucinaciones críticas en el chatbot citando leyes derogadas como si estuvieran activas), se asocia un **Trigger de Base de Datos AFTER**.
 - En el momento en que se inserta o actualiza un registro en **RelacionDocumental** bajo las condiciones de que el tipo de relación sea "Deroga" (ID 2 en el catálogo) y el flag **verificada** sea **TRUE****, **PostgreSQL ejecuta de forma inmediata una actualización en cascada sobre la tabla maestra Documento. Cambia automáticamente su idEstadoVigencia a "Derogado", estampa la fecha de efecto enderogacion y conmuta isActive = FALSE**, sacando instantáneamente el documento del contexto de búsqueda del motor RAG en estricto cumplimiento con la **Ley 21.663** de Ciberseguridad en Chile e **ISO 27001**.

- **Nota sobre la Sintaxis Utilizada:** En PostgreSQL, un **UNIQUE INDEX** y una **UNIQUE CONSTRAINT** son equivalentes a nivel de rendimiento y restricciones físicas en el disco. En la arquitectura de este sistema se opta explícitamente por la declaración de **CREATE UNIQUE INDEX** debido a su mayor flexibilidad operativa en entornos de producción (permite modificaciones avanzadas como cláusulas **INCLUDE**, indexación condicional **WHERE** o construcciones en caliente utilizando **CONCURRENTLY** sin bloquear lecturas de las aplicaciones conectadas).
- **Justificación del Trigger BEFORE para Ciclos:** Los **CHECK constraints** tradicionales no poseen estado semántico ni acceso a otras filas de la tabla, por lo que son matemáticamente incapaces de resolver la transitividad indirecta. El uso de un disparador **BEFORE** optimiza el uso de I/O y CPU, ya que detiene y revierte el bloque completo *antes* de que PostgreSQL intente escribir las páginas de datos en el almacenamiento o recalcular los índices físicos de la tabla.
- **Control de Falsos Positivos en Updates:** La función almacena la cláusula **idRelacion <> COALESCE(NEW.idRelacion, -1)**. Esto garantiza que si un usuario está editando una fila existente para corregir un metadato menor (como el texto de la evidencia o la confianza), el árbol recursivo ignore la versión previa que está actualmente guardada en el disco, evitando colisiones internas y bloqueos erróneos sobre modificaciones legítimas.

4. Tabla: **TipoRelacion**

Catálogo maestro que estandariza las acciones del motor relacional.

- **Propósito:** Almacena valores fijos, genéricos y reutilizables que configuran el backend para realizar filtrados masivos. Ejemplos en Base de Datos: 1 -> 'MODIFICA', 2 -> 'DEROGA', 3 -> 'COMPLEMENTA' -> , 4 -> 'REEMPLAZA'.

5. Tabla: **RespaldoLegal**

Entidad fundamental para la trazabilidad, auditoría y el cumplimiento normativo e institucional de la universidad (ISO 27001 / Ley 21663).

- **Definición Operativa:** Almacena el sustento formal, decreto, acta o acuerdo oficial que autoriza jurídicamente que un documento afecte a otro.
- **Optimización en Tercera Forma Normal (3FN):** Evita la redundancia de texto. Si en una sesión de Consejo Superior (ej. Sesión N° 337, Acuerdo N° 1493) se aprueba una reforma que afecta a 10 reglamentos distintos, el texto largo del acta se escribe **una sola vez** en esta tabla. Las 10 filas de la tabla **RelacionDocumental** apuntarán al mismo **idRespaldoLegal**, optimizando drásticamente la memoria RAM de PostgreSQL.
- **Blindaje de No Repudio:** Ante cualquier impugnación, el sistema recupera inmediatamente el respaldo legal exacto que autorizó el cambio de estado de una norma.

MÓDULO 3: TAXONOMÍAS Y ESTRUCTURAS JERÁRQUICAS (PADRE-HIJO)

Para garantizar búsquedas instantáneas y cumplir con la **3FN estricta**, el modelo implementa dependencias directas en cascada para categorizar los datos:

6. Relación: **TipoPrograma** → **NivelAcademico** → **Carrera**

Diseño de catálogo jerárquico para segmentar de manera efectiva la oferta de la institución.

- **TipoPrograma (Padre / Macro-Filtro)**: Almacena grupos macro como *'Pregrado'*, *'Postgrado'* o *'Educación Continua'*.
- **NivelAcademico (Hijo / Filtro Específico)**: Se conecta a su respectivo padre (**idTipoPrograma**) y almacena opciones específicas: *'Técnico'*, *'Ingeniería Civil'*, *'Magíster'*, *'Doctorado'*, *'Diplomado'*.
- **Carrera**: Entidad final dependiente que asocia el nivel (**idNivel**) con la unidad correspondiente (**idDepartamento**), conteniendo el **codigoCarrera** y el **nombreCarrera**.
- **Lógica de Filtros en Cascada**: Si un usuario selecciona el filtro macro *"Postgrado"*, el backend realiza un **JOIN** relacional indexado y veloz para recuperar automáticamente los documentos asociados a los IDs de *Magíster* y *Doctorado*, excluyendo de forma inmediata los *Diplomados*.

7. Relación: **MacroCategoria** → **Categoria**

Estructura de dos niveles para indexar la naturaleza jurídica de los textos corporativos.

- **MacroCategoria**: Define las áreas generales (ej: **siglaMacro**).
- **Categoria**: Contiene subdivisiones operativas con su respectivo código numérico (**numCategoria**) enlazadas a la macroestructura.

8. Relación: **TipoArea** → **subAreas**

Estructura relacional para el organigrama institucional.

- **TipoArea**: Define la naturaleza de las divisiones administrativas.
- **subAreas**: Departamentos u órganos específicos que componen la universidad, con su respectiva **siglaSubArea**.

MÓDULO 4: DICCIONARIOS DE CONFIGURACIÓN Y CONTEXTO (LOOKUPS)

Tablas maestras de valores únicos diseñadas para estandarizar los datos introducidos en el núcleo:

- **TipoDocumento**: Define la categoría legal del archivo ('Decreto', 'Resolución', 'Reglamento').
- **EstadoVigencia**: Controla la validez temporal del documento ('Vigente', 'Derogado', 'Modificado').
- **TipoSesion**: Define estrictamente si la sesión celebrada por el cuerpo colegiado fue de carácter 'Ordinaria' o 'Extraordinaria'.
- **TipoDecision**: Clasifica la acción legal tomada por el cuerpo colegiado: 'Aprobación', 'Autorización', 'Toma de conocimiento' o 'Ratificación'.
- **Jornada**: Registros fijos de los regímenes horarios de estudio: 'Diurno', 'Vespertino' y/o 'Residencial'.
- **RollInstitucional**: Guarda los cargos, perfiles o públicos objetivos de la universidad a los que afecta o va dirigido un documento (ej: 'Alumno', 'Jefe de Carrera'), sirviendo como un filtro estratégico para el buscador.
- **BeneficioInterno**: Almacena los programas de apoyo, becas o convenios internos de la institución (**nombreBeneficio**).
- **SedeCampus**: Catálogo de recintos e infraestructura física de la institución.
- **Departamento**: Unidades académicas de la institución que dictan o administran las asignaturas.
- **TipoNombramiento**: Cargos oficiales fijos de la estructura orgánica universitaria ('Rector', 'Decano', 'Director de Departamento').
- **FuenteDeteccion**: Clasifica y audita la procedencia de las interacciones. Permite al sistema discriminar de manera inteligente la confianza depositada en la relación antes de alterar la vigencia de un documento en producción. ('Manual', 'Automática' (Regex), **nombreArchivo**)
- **TipoPagina**: evita el uso de texto libre para categorizar las páginas, estandarizando tipos y permitiendo filtrar de manera eficiente (por ejemplo, excluir páginas clasificadas como "Índice" o "Firmas" para evitar ruido en la recuperación semántica del motor RAG). ('Portada', 'Índice', 'Cuerpo Normativo', 'Anexo', 'Firma' y 'Timbres')

MÓDULO 5: MATRIZ DE RELACIONES MULTIVALOR (TABLAS PUENTE N:M)

Para dar soporte a la complejidad del entorno universitario y garantizar que las consultas se resuelvan en **microsegundos**, el modelo rompe las relaciones multivalor mediante tablas intermedias con **Llaves Primarias Compuestas**:

Tabla Puente	PK Compuesta (Atributos)	Lógica de Negocio y Justificación Técnica Incorporada en el Diagrama Desconocido
DocumentoNombramiento	(idDocumento, idTipoNombramiento)	Eficiencia del Filtro (Indexación Natural): PostgreSQL crea un índice agrupado sobre ambas columnas. Al buscar por el cargo "Director de Departamento", el motor salta directamente a los documentos en microsegundos sin escanear toda la tabla. Cumple la 3FN estricta: si un cargo cambia de nombre, solo se altera una fila en TipoNombramiento y se refleja en toda la aplicación. Soporta que un documento registre un único nombramiento (ej. el Rector) o múltiples (ej. la lista de Directores electos del año).

DocumentoJornada y DocumentoNivel	(idDocumento, idJornada) y (idDocumento, idNivel)	Un solo documento corporativo (como un <i>Reglamento de Matrícula</i>) suele aplicar a varias jornadas y niveles académicos simultáneamente. Separar estos conceptos en tablas puente con PK compuestas garantiza Independencia Absoluta de Filtros. Si se busca un documento con los filtros "<i>Ingeniería Civil</i>" y "<i>Vespertino</i>", el backend realiza una intersección directa de IDs, eliminando el riesgo de confundir normativas de postgrado diurnas con carreras de pregrado vespertinas o de residencia. Permite una Flexibilidad Total a Futuro: si se crean nuevas jornadas o niveles, la estructura de la base de datos no sufre alteraciones.
AreaEmisora	(idAreaEmisora) (Nota: utiliza subAreas)	Mapea el o los órganos de la administración universitaria (subAreas) que emitieron y firmaron de forma conjunta un documento.

DocumentoRol	(idDocumento, idRol)	Vincula de manera directa a qué estamentos institucionales (alumnos, académicos, funcionarios) afecta u otorga derechos el documento.
DocumentoBeneficio	(idDocumento, idBeneficio)	Conecta las normativas oficiales con los beneficios de la universidad, facilitando los filtros en el portal web de asistencia estudiantil.
DocumentoSedeCampus	(idDocumento, idRecintoUniversitario)	Delimita la aplicación del documento a una sede o campus específico, aislando decretos de aplicación local.
DocumentoDepartamento	(idDocumento, idDepartamento)	Relaciona qué facultades o departamentos académicos se ven regulados o afectados por la emisión de la norma.
DocumentoCarrera	(idDocumento, idCarrera)	Permite indexar normativas, mallas o resoluciones específicas aplicables a una o más carreras en particular.

Notas de Integridad de Datos para el Administrador:

1. **Políticas de Borrado:** Todas las tablas puente (Módulo 5), la tabla `DocumentoChunk` y la tabla `RelacionDocumental` implementan restricciones de integridad foránea con cláusulas `ON DELETE CASCADE` hacia la tabla maestra `Documento`. Esto garantiza que si un documento es retirado, no queden datos huérfanos ni registros basura en producción.
2. **Protección de Diccionarios:** Las tablas del Módulo 3 y Módulo 4 hacia la tabla `Documento` utilizan `ON DELETE RESTRICT`. Esto impide la eliminación accidental de un catálogo activo que rompería el linaje histórico exigido por las normas internacionales de auditoría de TI.

Script de creación de las tablas: (ACTUALIZADO 16/jul 23:02)

```
-- =====
-- PROYECTO G.I.R.U. (2026) - SCRIPT DDL COMPLETO DE BASE DE DATOS (v3)
-- =====

BEGIN;

-- Habilitación obligatoria de extensiones para Inteligencia Artificial
CREATE EXTENSION IF NOT EXISTS vector;

-- =====
-- 1. CATÁLOGOS BASE (ON DELETE RESTRICT)
-- =====

CREATE TABLE TipoArea (
    idTipoArea SERIAL PRIMARY KEY,
    nombreArea VARCHAR(100) NOT NULL,
    siglaArea VARCHAR(20) UNIQUE NOT NULL
);

CREATE TABLE MacroCategoria (
    idMacroCategoria SERIAL PRIMARY KEY,
    siglaMacro CHAR(2) UNIQUE NOT NULL,
    nombreCategoria VARCHAR(100) NOT NULL
);

CREATE TABLE TipoDocumento (
    idTipoDocumento SERIAL PRIMARY KEY,
    nombreTipoDocumento VARCHAR(100) UNIQUE NOT NULL
);

CREATE TABLE EstadoVigencia (
    idEstadoVigencia SERIAL PRIMARY KEY,
    estadoVigencia VARCHAR(50) UNIQUE NOT NULL
);

CREATE TABLE RespaldoLegal (
    idRespaldoLegal SERIAL PRIMARY KEY,
    respaldoLegal VARCHAR(255) NOT NULL
);

CREATE TABLE RolInstitucional (
    idRol SERIAL PRIMARY KEY,
```



```

    nombreRol VARCHAR(100) UNIQUE NOT NULL
);

CREATE TABLE BeneficioInterno (
    idBeneficio SERIAL PRIMARY KEY,
    nombreBeneficio VARCHAR(150) UNIQUE NOT NULL
);

CREATE TABLE SedeCampus (
    idRecintoUniversitario SERIAL PRIMARY KEY,
    nombreRecinto VARCHAR(100) UNIQUE NOT NULL
);

CREATE TABLE TipoNombramiento (
    idTipoNombramiento SERIAL PRIMARY KEY,
    nombreNombramiento VARCHAR(100) UNIQUE NOT NULL
);

CREATE TABLE Jornada (
    idJornada SERIAL PRIMARY KEY,
    nombreJornada VARCHAR(50) UNIQUE NOT NULL
);

CREATE TABLE TipoPrograma (
    idTipoPrograma SERIAL PRIMARY KEY,
    nombreTipoPrograma VARCHAR(100) UNIQUE NOT NULL
);

CREATE TABLE Departamento (
    idDepartamento SERIAL PRIMARY KEY,
    nombreDepartamento VARCHAR(150) UNIQUE NOT NULL
);

CREATE TABLE TipoSesion (
    idTipoSesion SERIAL PRIMARY KEY,
    tipoSesion VARCHAR(50) UNIQUE NOT NULL
);

CREATE TABLE TipoDecision (
    idTipoDecision SERIAL PRIMARY KEY,
    tipoDecision VARCHAR(50) UNIQUE NOT NULL
);

CREATE TABLE TipoRelacion (
    idTipoRelacion SERIAL PRIMARY KEY,

```

```

    tipoRelacion VARCHAR(50) UNIQUE NOT NULL
);

CREATE TABLE FuenteDeteccion (
    idFuente SERIAL PRIMARY KEY,
    nomFuente VARCHAR(60) UNIQUE NOT NULL
);

CREATE TABLE TipoPagina (
    idTipoPagina SERIAL PRIMARY KEY,
    tipoPagina VARCHAR(50) UNIQUE NOT NULL
);

-- =====
-- 2. ENTIDADES JERÁRQUICAS (PADRE-HIJO)
-- =====

CREATE TABLE SubArea (
    idSubArea SERIAL PRIMARY KEY,
    idTipoArea INT NOT NULL REFERENCES TipoArea(idTipoArea) ON DELETE RESTRICT,
    nombre VARCHAR(150) NOT NULL,
    siglaSubArea VARCHAR(50) UNIQUE NOT NULL
);

CREATE TABLE Categoria (
    idCategoria SERIAL PRIMARY KEY,
    idMacroCategoria INT NOT NULL REFERENCES MacroCategoria(idMacroCategoria) ON DELETE RESTRICT,
    numCategoria VARCHAR(20) UNIQUE NOT NULL,
    nombreCategoria VARCHAR(150) NOT NULL
);

CREATE TABLE NivelAcademico (
    idNivel SERIAL PRIMARY KEY,
    idTipoPrograma INT NOT NULL REFERENCES TipoPrograma(idTipoPrograma) ON DELETE RESTRICT,
    nombreNivel VARCHAR(100) NOT NULL
);

CREATE TABLE Carrera (
    idCarrera SERIAL PRIMARY KEY,
    idNivel INT NOT NULL REFERENCES NivelAcademico(idNivel) ON DELETE RESTRICT,
    idDepartamento INT NOT NULL REFERENCES Departamento(idDepartamento) ON DELETE RESTRICT,
    codigoCarrera VARCHAR(50) UNIQUE,
    nombreCarrera VARCHAR(150) NOT NULL
);

-- =====

```

```

-- 3. NÚCLEO TRANSACCIONAL
-- =====
CREATE TABLE Documento (
    idDocumento SERIAL PRIMARY KEY,
    idTipoDocumento INT NOT NULL REFERENCES TipoDocumento(idTipoDocumento) ON DELETE RESTRICT,
    idCategoria INT NOT NULL REFERENCES Categoria(idCategoria) ON DELETE RESTRICT,
    idTipoSesion INT REFERENCES TipoSesion(idTipoSesion) ON DELETE RESTRICT,
    idEstadoVigencia INT NOT NULL REFERENCES EstadoVigencia(idEstadoVigencia) ON DELETE RESTRICT,
    idTipoDecision INT REFERENCES TipoDecision(idTipoDecision) ON DELETE RESTRICT,
    numSesion INT,
    numAcuerdo INT,
    numero VARCHAR(50) UNIQUE NOT NULL,
    titulo VARCHAR(255) NOT NULL,
    nomMetaDato VARCHAR(255) NOT NULL UNIQUE,
    descripcion TEXT,
    urlArchivoOriginalS3 VARCHAR(500) NOT NULL,
    urlOcrS3 VARCHAR(500) NOT NULL,
    cant_paginas INT NOT NULL CHECK (cant_paginas > 0),
    creacion DATE NOT NULL,
    derogacion DATE,
    aplicacionInmediata BOOLEAN,
    isActive BOOLEAN DEFAULT TRUE
);

-- =====
-- 4. INTELIGENCIA ARTIFICIAL (RAG)
-- =====
CREATE TABLE DocumentoChunk (
    idChunk SERIAL PRIMARY KEY,
    idDocumento INT NOT NULL REFERENCES Documento(idDocumento) ON DELETE CASCADE,
    idTipoPagina INT NOT NULL REFERENCES TipoPagina(idTipoPagina) ON DELETE RESTRICT,
    numeroPagina INT NOT NULL,
    nombreTitulo VARCHAR(255), -- Nullable para flexibilidad del pipeline
    numeroArticulo VARCHAR(50), -- Acorde a especificación
    numeroInciso VARCHAR(50), -- Acorde a especificación
    secuencia INT NOT NULL,
    textoFragmento TEXT NOT NULL,
    indiceLexico TSVECTOR GENERATED ALWAYS AS (to_tsvector('spanish', COALESCE(textoFragmento, '')))
STORED,
    hashSha256 CHAR(64) NOT NULL,
    embedding VECTOR(1024), -- Parametrizado para BGE-M3
    isActive BOOLEAN DEFAULT TRUE
);

-- Optimización de almacenamiento en disco para fragmentos de texto

```

```

ALTER TABLE DocumentoChunk ALTER COLUMN textoFragmento SET COMPRESSION lz4;

-- =====
-- 5. TRAZABILIDAD (LINAJE JURÍDICO)
-- =====

CREATE TABLE RelacionDocumental (
    idRelacion SERIAL PRIMARY KEY,
    idFuenteDeteccion INT NOT NULL REFERENCES FuenteDeteccion(idFuente) ON DELETE RESTRICT,
    idRespaldoLegal INT NOT NULL REFERENCES RespaldoLegal(idRespaldoLegal) ON DELETE RESTRICT,
    idTipoRelacion INT NOT NULL REFERENCES TipoRelacion(idTipoRelacion) ON DELETE RESTRICT,
    idDocumentoOrigen INT NOT NULL REFERENCES Documento(idDocumento) ON DELETE CASCADE,
    idDocumentoDestino INT NOT NULL REFERENCES Documento(idDocumento) ON DELETE CASCADE,
    detalleModificacion VARCHAR(255),
    confianza NUMERIC(3,2) DEFAULT 1.00 CHECK (confianza >= 0.00 AND confianza <= 1.00),
    verificada BOOLEAN DEFAULT FALSE,
    textoEvidencia TEXT,
    fechaEfecto DATE NOT NULL,
    CONSTRAINT chk_origen_destino CHECK (idDocumentoOrigen <> idDocumentoDestino)
);

-- =====
-- 6. TABLAS PUENTE (RELACIONES N:M) CON LLAVES COMPUESTAS (MÓDULO 5 COMPLETO)
-- =====

CREATE TABLE AreaEmisora (
    idAreaEmisora SERIAL PRIMARY KEY,
    idDocumento INT NOT NULL REFERENCES Documento(idDocumento) ON DELETE CASCADE,
    idAreaAdministrativa INT NOT NULL REFERENCES SubArea(idSubArea) ON DELETE RESTRICT
);

CREATE TABLE DocumentoNombramiento (
    idDocumento INT NOT NULL REFERENCES Documento(idDocumento) ON DELETE CASCADE,
    idTipoNombramiento INT NOT NULL REFERENCES TipoNombramiento(idTipoNombramiento) ON DELETE CASCADE,
    PRIMARY KEY (idDocumento, idTipoNombramiento)
);

CREATE TABLE DocumentoJornada (
    idDocumento INT NOT NULL REFERENCES Documento(idDocumento) ON DELETE CASCADE,
    idJornada INT NOT NULL REFERENCES Jornada(idJornada) ON DELETE CASCADE,
    PRIMARY KEY (idDocumento, idJornada)
);

CREATE TABLE DocumentoNivel (
    idDocumento INT NOT NULL REFERENCES Documento(idDocumento) ON DELETE CASCADE,
    idNivel INT NOT NULL REFERENCES NivelAcademico(idNivel) ON DELETE CASCADE,
    PRIMARY KEY (idDocumento, idNivel)
);

```

```

);

CREATE TABLE DocumentoRol (
    idDocumento INT NOT NULL REFERENCES Documento(idDocumento) ON DELETE CASCADE,
    idRol INT NOT NULL REFERENCES RolInstitucional(idRol) ON DELETE CASCADE,
    PRIMARY KEY (idDocumento, idRol)
);

CREATE TABLE DocumentoBeneficio (
    idDocumento INT NOT NULL REFERENCES Documento(idDocumento) ON DELETE CASCADE,
    idBeneficio INT NOT NULL REFERENCES BeneficioInterno(idBeneficio) ON DELETE CASCADE,
    PRIMARY KEY (idDocumento, idBeneficio)
);

CREATE TABLE DocumentoSedeCampus (
    idDocumento INT NOT NULL REFERENCES Documento(idDocumento) ON DELETE CASCADE,
    idRecintoUniversitario INT NOT NULL REFERENCES SedeCampus(idRecintoUniversitario) ON DELETE CASCADE,
    PRIMARY KEY (idDocumento, idRecintoUniversitario)
);

CREATE TABLE DocumentoDepartamento (
    idDocumento INT NOT NULL REFERENCES Documento(idDocumento) ON DELETE CASCADE,
    idDepartamento INT NOT NULL REFERENCES Departamento(idDepartamento) ON DELETE CASCADE,
    PRIMARY KEY (idDocumento, idDepartamento)
);

CREATE TABLE DocumentoCarrera (
    idDocumento INT NOT NULL REFERENCES Documento(idDocumento) ON DELETE CASCADE,
    idCarrera INT NOT NULL REFERENCES Carrera(idCarrera) ON DELETE CASCADE,
    PRIMARY KEY (idDocumento, idCarrera)
);

-- =====
-- 7. ÍNDICES DE BÚSQUEDA HÍBRIDA Y RENDIMIENTO
-- =====

CREATE INDEX idx_doc_numero ON Documento(numero);
CREATE INDEX idx_chunk_lexico ON DocumentoChunk USING GIN (indiceLexico);

-- Índice HNSW de alta precisión para las búsquedas vectoriales conversacionales (BGE-M3)
CREATE INDEX idx_chunk_embedding_hnsw
ON DocumentoChunk USING hnsw (embedding vector_cosine_ops);

-- Índice Único de negocio para evitar duplicados lógicos en la trazabilidad
CREATE UNIQUE INDEX idx_rel_unica
ON RelacionDocumental(idDocumentoOrigen, idDocumentoDestino, idTipoRelacion);

```

```

CREATE INDEX idx_chunk_lexico ON DocumentoChunk USING GIN (indiceLexico);

-- =====
-- 8. AUTOMATIZACIÓN (TRIGGER DE SINCRONIZACIÓN DE VIGENCIA SEMÁNTICA)
-- =====

CREATE OR REPLACE FUNCTION fn_sincronizar_vigencia_derogacion()
RETURNS TRIGGER AS $$
BEGIN
    -- Evalúa si la relación fue validada por un humano y si su tipo asociado es "Deroga" consultando la
    -- tabla catálogo
    IF NEW.verificada = TRUE AND EXISTS (
        SELECT 1
        FROM TipoRelacion
        WHERE idTipoRelacion = NEW.idTipoRelacion
        AND UPPER(tipoRelacion) = 'DEROGA'
    ) THEN
        UPDATE Documento
        SET idEstadoVigencia = (SELECT idEstadoVigencia FROM EstadoVigencia WHERE UPPER(estadoVigencia)
= 'DEROGADO'), -- Obtiene dinámicamente el ID del estado 'Derogado'
        derogacion = NEW.fechaEfecto, -- Sincroniza la fecha oficial del cambio de estado
        isActive = FALSE -- Apaga su visibilidad para búsquedas RAG
        WHERE idDocumento = NEW.idDocumentoDestino;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_relacion_documental_vigencia
AFTER INSERT OR UPDATE ON RelacionDocumental
FOR EACH ROW
EXECUTE FUNCTION fn_sincronizar_vigencia_derogacion();

--OTRO TRIGGER (GRAFO)

-- 1. Crear la función procedimental que audita el grafo en tiempo real
CREATE OR REPLACE FUNCTION fn_prevenir_ciclos_documentales()
RETURNS TRIGGER AS $$
DECLARE
    v_ciclo_detectado BOOLEAN := FALSE;
BEGIN
    -- Control de seguridad inmediato (Bucle A -> A)
    IF NEW.idDocumentoOrigen = NEW.idDocumentoDestino THEN
        RAISE EXCEPTION 'Violación de Integridad: Un documento no puede relacionarse consigo mismo
(Ciclo inmediato).';
    END IF;

```

```

-- Evaluamos únicamente si es una inserción nueva o si se están modificando los nodos de una
relación existente
IF TG_OP = 'INSERT' OR (TG_OP = 'UPDATE' AND (OLD.idDocumentoOrigen <> NEW.idDocumentoOrigen OR
OLD.idDocumentoDestino <> NEW.idDocumentoDestino)) THEN

-- Usamos una Expresión de Tabla Común (CTE) Recursiva para navegar el grafo río abajo
WITH RECURSIVE camino AS (
-- ANCLA: Comenzamos buscando todas las relaciones que nacen desde el DESTINO de nuestra
nueva fila
SELECT idDocumentoDestino AS nodo_actual
FROM RelacionDocumental
WHERE idDocumentoOrigen = NEW.idDocumentoDestino
-- CRÍTICO: Si es un UPDATE, excluimos la fila actual para que la recursividad no use el
enlace viejo del disco
AND idRelacion <> COALESCE(NEW.idRelacion, -1)
UNION
-- PASO RECURSIVO: Seguimos saltando de destino en destino para mapear toda la cadena transitiva
SELECT r.idDocumentoDestino
FROM RelacionDocumental r
INNER JOIN camino c ON r.idDocumentoOrigen = c.nodo_actual
WHERE r.idRelacion <> COALESCE(NEW.idRelacion, -1)
)
-- Si en todo el árbol de alcance logramos encontrar el ID del Origen, significa que el camino
se cierra (HAY CICLO)
SELECT EXISTS (
SELECT 1 FROM camino WHERE nodo_actual = NEW.idDocumentoOrigen
) INTO v_ciclo_detectado;
-- Si se detecta el bucle, disparamos una excepción con código de estado personalizado y
revertimos la transacción
IF v_ciclo_detectado THEN
RAISE EXCEPTION 'Violación de Grafo Acíclico (DAG): La relación propuesta entre el documento
origen (ID: %) y destino (ID: %) genera un bucle circular infinito (Transitividad detectada).',
NEW.idDocumentoOrigen, NEW.idDocumentoDestino;
END IF;
END IF;
RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- 2. Enlazar la función como un trigger de tipo BEFORE
-- Debe ser BEFORE para impedir el gasto de CPU/I/O de escritura e indexación si la regla se rompe
CREATE TRIGGER trg_relacion_documental_prevenccion_ciclos
BEFORE INSERT OR UPDATE ON RelacionDocumental
FOR EACH ROW

```

```
EXECUTE FUNCTION fn_prevenir_ciclos_documentales();
COMMIT;
```

Script para poblar la base de datos, contiene datos ficticios y de ejemplo (**Posee una limpieza de seguridad inicial**) (**ACTUALIZADO, NO VERIFICADO: 16/jul 23:09**):

```
-- =====
-- SCRIPT DE POBLADO MASIVO Y PRUEBAS RAG - PROYECTO G.I.R.U. 2026
-- Volumen: 25 Documentos, Catálogos completos, Chunks IA y Trazabilidad DAG
-- =====

BEGIN;

-- 0. LIMPIEZA DE SEGURIDAD (Para reiniciar contadores a 1)
TRUNCATE TABLE
    TipoArea, MacroCategoria, TipoDocumento, EstadoVigencia, RespaldoLegal,
    RolInstitucional, BeneficioInterno, SedeCampus, TipoNombramiento, Jornada,
    TipoPrograma, Departamento, TipoSesion, TipoDecision, TipoRelacion,
    FuenteDeteccion, TipoPagina CASCADE;

-- =====
-- 1. POBLADO DE CATÁLOGOS BASE (LOOKUP TABLES)
-- =====

INSERT INTO TipoArea (nombreArea, siglaArea) VALUES
('Académica', 'ACAD'), ('Administración y Finanzas', 'ADM'), ('Asuntos Estudiantiles', 'AEST'),
('Investigación y Postgrado', 'INV-POST'), ('Vinculación con el Medio', 'VVM');

INSERT INTO MacroCategoria (siglaMacro, nombreCategoria) VALUES
('GI', 'Gestión Institucional'), ('DP', 'Docencia de Pregrado'), ('DO', 'Docencia de Postgrado'),
('VM', 'Vinculación con el Medio'), ('IN', 'Investigación');

INSERT INTO TipoDocumento (nombreTipoDocumento) VALUES
('Reglamento'), ('Decreto Exento'), ('Resolución Rectoría'), ('Instructivo'), ('Normativa');

INSERT INTO EstadoVigencia (estadoVigencia) VALUES
('Vigente'), ('Derogado'), ('Modificado'), ('En Revisión');

INSERT INTO RespaldoLegal (respaldoLegal) VALUES
('Acta Consejo Superior N° 100'), ('Acta Consejo Académico N° 45'), ('Decreto Institucional N°
01/2020'),
('Resolución Contraloría N° 15'), ('Ley de Educación Superior 21.091');

INSERT INTO RolInstitucional (nombreRol) VALUES
('Alumno'), ('Jefe de Carrera'), ('Docente'), ('Director de Departamento'), ('Decano'),
```



```

('Administrativo');

INSERT INTO BeneficioInterno (nombreBeneficio) VALUES
('Beca Alimentación'), ('Beca Arancel'), ('Fondo de Investigación'), ('Subsidio Movilidad'), ('Apoyo Tecnológico');

INSERT INTO SedeCampus (nombreRecinto) VALUES
('Casa Central (Valparaíso)'), ('Campus San Joaquín'), ('Campus Vitacura'), ('Sede Viña del Mar'), ('Sede Concepción');

INSERT INTO TipoNombramiento (nombreNombramiento) VALUES
('Rector'), ('Decano'), ('Director General'), ('Jefe de Carrera'), ('Secretario General');

INSERT INTO Jornada (nombreJornada) VALUES ('Diurno'), ('Vespertino'), ('Residencial/Intensivo');

INSERT INTO TipoPrograma (nombreTipoPrograma) VALUES ('Pregrado'), ('Postgrado'), ('Educación Continua');

INSERT INTO Departamento (nombreDepartamento) VALUES
('Informática'), ('Electrónica'), ('Mecánica'), ('Industrias'), ('Matemática'), ('Física');

INSERT INTO TipoSesion (tipoSesion) VALUES ('Ordinaria'), ('Extraordinaria'), ('Solemne');

INSERT INTO TipoDecision (tipoDecision) VALUES ('Aprobación'), ('Autorización'), ('Toma de Conocimiento'), ('Rechazo');

INSERT INTO TipoRelacion (tipoRelacion) VALUES ('Modifica'), ('Deroga'), ('Complementa'), ('Reemplaza');

INSERT INTO FuenteDeteccion (nomFuente) VALUES ('Ingreso Manual'), ('Detección Automática (Regex)'), ('Análisis de Nombre de Archivo');

INSERT INTO TipoPagina (nombreTipoPagina) VALUES ('Portada'), ('Índice'), ('Cuerpo Normativo'), ('Anexo'), ('Firmas y Timbres');

-- =====
-- 2. POBLADO DE ENTIDADES JERÁRQUICAS
-- =====

INSERT INTO subAreas (idTipoArea, nombre, siglaSubArea) VALUES
(1, 'Secretaría General', 'SEC-GEN'), (2, 'Dirección de Finanzas', 'DIR-FIN'),
(3, 'Relaciones Estudiantiles', 'RREL-EST'), (4, 'Dirección de Postgrado', 'DIR-POST'), (5, 'Dirección de Innovación', 'DIR-INV');

INSERT INTO Categoria (idMacroCategoria, numCategoria, nombreCategoria) VALUES

```

```

(1, 'GI-01', 'Estructura Orgánica'), (1, 'GI-02', 'Presupuesto'),
(2, 'DP-01', 'Régimen Curricular Pregrado'), (2, 'DP-02', 'Admisión Pregrado'),
(3, 'DO-01', 'Normativas de Magíster'), (3, 'DO-02', 'Becas de Postgrado'),
(4, 'VM-01', 'Convenios Internacionales'), (4, 'VM-02', 'Prácticas y Titulación'),
(5, 'IN-01', 'Propiedad Intelectual'), (5, 'IN-02', 'Laboratorios');

INSERT INTO NivelAcademico (idTipoPrograma, nombreNivel) VALUES
(1, 'Técnico Universitario'), (1, 'Ingeniería Civil'), (2, 'Magíster'), (2, 'Doctorado'), (3,
'Diplomado');

INSERT INTO Carrera (idNivel, idDepartamento, codigoCarrera, nombreCarrera) VALUES
(2, 1, 'INF-CIV-01', 'Ingeniería Civil Informática'), (1, 2, 'ELC-TEC-01', 'Técnico en
Electrónica'),
(3, 1, 'INF-MAG-01', 'Magíster en Informática'), (4, 4, 'IND-DOC-01', 'Doctorado en Industrias');

-- =====
-- 3. NÚCLEO TRANSACCIONAL (25 DOCUMENTOS ESTRATÉGICOS)
-- =====

INSERT INTO Documento (idTipoDocumento, idCategoria, idTipoSesion, idEstadoVigencia, idTipoDecision,
numSesion, numAcuerdo, numero, nomMetaDato, titulo, descripcion, urlArchivoOriginalS3, urlOcrS3,
cant_paginas, creacion, aplicacionInmediata, isActive) VALUES
-- Gestión Institucional (GI)
(1, 1, 1, 1, 1, 101, 10, 'DEC-001', 'GI-01-0001', 'Estatuto Orgánico Original', 'Estatuto
fundacional.', 's3://bucket/d1.pdf', 's3://bucket/d1.json', 50, '1990-01-01', TRUE, TRUE),
(2, 1, 1, 1, 1, 102, 11, 'DEC-002', 'GI-01-0002', 'Creación de Vicerrectorías', 'Ajuste
estructural.', 's3://bucket/d2.pdf', 's3://bucket/d2.json', 12, '1995-05-15', TRUE, TRUE),
(3, 2, 1, 1, 2, 103, 12, 'RES-001', 'GI-02-0003', 'Política Presupuestaria', 'Directrices
financieras.', 's3://bucket/d3.pdf', 's3://bucket/d3.json', 8, '2000-03-10', FALSE, TRUE),
(4, 2, 1, 1, 1, 104, 13, 'INS-001', 'GI-02-0004', 'Instructivo de Compras', 'Licitaciones
internas.', 's3://bucket/d4.pdf', 's3://bucket/d4.json', 25, '2005-08-20', TRUE, TRUE),
(1, 1, 2, 1, 1, 105, 14, 'DEC-003', 'GI-01-0005', 'Reglamento de Contraloría', 'Auditoría interna.',
's3://bucket/d5.pdf', 's3://bucket/d5.json', 15, '2010-11-11', TRUE, TRUE),

-- Docencia Pregrado (DP)
(1, 3, 1, 1, 1, 201, 20, 'DEC-100', 'DP-01-0006', 'Reglamento Curricular Pregrado', 'Normas de
alumnos.', 's3://bucket/d6.pdf', 's3://bucket/d6.json', 40, '2015-01-15', TRUE, TRUE),
(2, 4, 1, 1, 1, 202, 21, 'DEC-101', 'DP-02-0007', 'Vías de Admisión Especial', 'Cupos deportivos.',
's3://bucket/d7.pdf', 's3://bucket/d7.json', 10, '2016-04-12', TRUE, TRUE),
(3, 3, 1, 1, 2, 203, 22, 'RES-102', 'DP-01-0008', 'Reajuste de Aranceles', 'Valores de matrícula.',
's3://bucket/d8.pdf', 's3://bucket/d8.json', 5, '2017-09-05', FALSE, TRUE),
(4, 3, 1, 1, 1, 204, 23, 'INS-103', 'DP-01-0009', 'Instructivo de Titulación', 'Procesos de
memoria.', 's3://bucket/d9.pdf', 's3://bucket/d9.json', 18, '2018-02-28', TRUE, TRUE),
(1, 4, 2, 1, 1, 205, 24, 'DEC-104', 'DP-02-0010', 'Reglamento de Inclusión', 'Admisión mujeres
STEM.', 's3://bucket/d10.pdf', 's3://bucket/d10.json', 22, '2019-12-01', TRUE, TRUE),

```

```
-- Docencia Postgrado (D0)
(1, 5, 1, 1, 1, 301, 30, 'DEC-200', 'DO-01-0011', 'Reglamento General de Postgrado', 'Normas de
Magíster.', 's3://bucket/d11.pdf', 's3://bucket/d11.json', 35, '2012-06-15', TRUE, TRUE),
(2, 6, 1, 1, 1, 302, 31, 'DEC-201', 'DO-02-0012', 'Becas de Manutención', 'Fondos para doctorado.',
's3://bucket/d12.pdf', 's3://bucket/d12.json', 14, '2013-08-22', TRUE, TRUE),
(3, 5, 1, 1, 2, 303, 32, 'RES-202', 'DO-01-0013', 'Criterios de Graduación', 'Exigencias de tesis.',
's3://bucket/d13.pdf', 's3://bucket/d13.json', 7, '2014-11-30', FALSE, TRUE),
(4, 6, 1, 1, 1, 304, 33, 'INS-203', 'DO-02-0014', 'Instructivo Postulación Becas', 'Fechas y
plazos.', 's3://bucket/d14.pdf', 's3://bucket/d14.json', 9, '2015-05-10', TRUE, TRUE),
(1, 5, 2, 1, 1, 305, 34, 'DEC-204', 'DO-01-0015', 'Reglamento de Doctorados', 'Exigencias
ISI/Scopus.', 's3://bucket/d15.pdf', 's3://bucket/d15.json', 28, '2020-10-05', TRUE, TRUE),

-- Vinculación (VM) e Investigación (IN)
(1, 7, 1, 1, 1, 401, 40, 'DEC-300', 'VM-01-0016', 'Reglamento de Intercambio', 'Movilidad
internacional.', 's3://bucket/d16.pdf', 's3://bucket/d16.json', 16, '2010-02-14', TRUE, TRUE),
(2, 8, 1, 1, 1, 402, 41, 'DEC-301', 'VM-02-0017', 'Normativa de Prácticas', 'Seguros y horas.',
's3://bucket/d17.pdf', 's3://bucket/d17.json', 11, '2011-04-18', TRUE, TRUE),
(1, 9, 1, 1, 1, 501, 50, 'DEC-400', 'IN-01-0018', 'Reglamento Propiedad Intelectual', 'Patentes y
spin-offs.', 's3://bucket/d18.pdf', 's3://bucket/d18.json', 30, '2018-07-07', TRUE, TRUE),
(2, 10, 1, 1, 1, 502, 51, 'DEC-401', 'IN-02-0019', 'Protocolo de Laboratorios', 'Seguridad
química.', 's3://bucket/d19.pdf', 's3://bucket/d19.json', 20, '2019-09-12', TRUE, TRUE),
(3, 9, 2, 1, 2, 503, 52, 'RES-402', 'IN-01-0020', 'Fondos Semilla', 'Asignación de capital.',
's3://bucket/d20.pdf', 's3://bucket/d20.json', 6, '2021-03-25', FALSE, TRUE),
(1, 3, 1, 1, 1, 601, 60, 'DEC-500', 'DP-01-0021', 'Reglamento de Ayudantías', 'Pago a alumnos.',
's3://bucket/d21.pdf', 's3://bucket/d21.json', 12, '2022-01-10', TRUE, TRUE),
(2, 1, 1, 1, 1, 602, 61, 'DEC-501', 'GI-01-0022', 'Código de Ética', 'Sanciones acoso.',
's3://bucket/d22.pdf', 's3://bucket/d22.json', 18, '2022-05-15', TRUE, TRUE),
(1, 5, 1, 1, 1, 603, 62, 'DEC-502', 'DO-01-0023', 'Cotutelas Internacionales', 'Doble grado.',
's3://bucket/d23.pdf', 's3://bucket/d23.json', 15, '2023-08-20', TRUE, TRUE),
(2, 2, 1, 1, 1, 604, 63, 'DEC-503', 'GI-02-0024', 'Fondo de Emergencia', 'Presupuesto COVID.',
's3://bucket/d24.pdf', 's3://bucket/d24.json', 8, '2020-04-01', TRUE, TRUE),

-- Documento 25: El Asesino (Deroga al Documento 1)
(1, 1, 3, 1, 1, 999, 99, 'DEC-999', 'GI-01-0025', 'Nuevo Estatuto Orgánico 2024', 'Reemplaza al del
90.', 's3://bucket/d25.pdf', 's3://bucket/d25.json', 60, '2024-01-01', TRUE, TRUE);

-- =====
-- 4. INTELIGENCIA ARTIFICIAL (25 CHUNKS VECTORIALES SIMULADOS)
-- Hashes exactos de 64 caracteres requeridos por la DB
-- =====

INSERT INTO DocumentoChunk (idDocumento, idTipoPagina, numeroPagina, secuencia, nombreTitulo,
numeroArticulo, numeroInciso, textoFragmento, hashSha256) VALUES
(1, 3, 2, 1, 'Título I', 'Artículo 1', 'Inciso 1', 'La Universidad se rige por el Consejo
```

Superior...', 'a1b2c3d4e5f60718293a4b5c6d7e8f90a1b2c3d4e5f60718293a4b5c6d7e8f90'),
(2, 3, 4, 1, 'Título II', 'Artículo 5', 'Inciso 2', 'Se crean las Vicerrectorías para apoyar al Rector...', 'b2c3d4e5f60718293a4b5c6d7e8f90a1b2c3d4e5f60718293a4b5c6d7e8f90a1'),
(3, 3, 1, 1, 'Título Único', 'Artículo 1', 'Inciso 1', 'El presupuesto anual debe ser visado en diciembre...', 'c3d4e5f60718293a4b5c6d7e8f90a1b2c3d4e5f60718293a4b5c6d7e8f90a1b2'),
(4, 3, 10, 2, 'Capítulo III', 'Artículo 14', 'Inciso 3', 'Toda compra sobre 500 UF requiere licitación...', 'd4e5f60718293a4b5c6d7e8f90a1b2c3d4e5f60718293a4b5c6d7e8f90a1b2c3'),
(5, 3, 5, 1, 'Título IV', 'Artículo 20', 'Inciso 1', 'La Contraloría interna auditará cada semestre...', 'e5f60718293a4b5c6d7e8f90a1b2c3d4e5f60718293a4b5c6d7e8f90a1b2c3d4'),
(6, 3, 12, 1, 'Título II', 'Artículo 27', 'Inciso 1', 'Un alumno reprobará si reprueba 3 veces el ramo...', 'f60718293a4b5c6d7e8f90a1b2c3d4e5f60718293a4b5c6d7e8f90a1b2c3d4e5'),
(7, 3, 3, 1, 'Capítulo I', 'Artículo 2', 'Inciso 1', 'Existen 50 cupos para deportistas destacados...', '0718293a4b5c6d7e8f90a1b2c3d4e5f60718293a4b5c6d7e8f90a1b2c3d4e5f6'),
(8, 3, 2, 2, 'Título Único', 'Artículo 4', 'Inciso 2', 'El arancel subirá según el IPC anual...', '18293a4b5c6d7e8f90a1b2c3d4e5f60718293a4b5c6d7e8f90a1b2c3d4e5f607'),
(9, 3, 15, 1, 'Capítulo V', 'Artículo 40', 'Inciso 1', 'La memoria requiere un profesor guía y un corref.', '293a4b5c6d7e8f90a1b2c3d4e5f60718293a4b5c6d7e8f90a1b2c3d4e5f60718'),
(10, 3, 8, 1, 'Título II', 'Artículo 12', 'Inciso 1', 'Se garantizará paridad en carreras STEM...', '3a4b5c6d7e8f90a1b2c3d4e5f60718293a4b5c6d7e8f90a1b2c3d4e5f6071829'),
(11, 3, 6, 1, 'Título III', 'Artículo 18', 'Inciso 1', 'El Magíster durará 4 semestres académicos...', '4b5c6d7e8f90a1b2c3d4e5f60718293a4b5c6d7e8f90a1b2c3d4e5f60718293a'),
(12, 3, 4, 1, 'Capítulo II', 'Artículo 5', 'Inciso 1', 'La beca cubre el 100% del arancel y mantención...', '5c6d7e8f90a1b2c3d4e5f60718293a4b5c6d7e8f90a1b2c3d4e5f60718293a4b'),
(13, 3, 2, 1, 'Título Único', 'Artículo 3', 'Inciso 2', 'Se exige una publicación Scopus para graduarse...', '6d7e8f90a1b2c3d4e5f60718293a4b5c6d7e8f90a1b2c3d4e5f60718293a4b5c'),
(14, 3, 1, 1, 'Título I', 'Artículo 1', 'Inciso 1', 'Las postulaciones abren el 15 de marzo de cada año', '7e8f90a1b2c3d4e5f60718293a4b5c6d7e8f90a1b2c3d4e5f60718293a4b5c6d'),
(15, 3, 20, 1, 'Título V', 'Artículo 50', 'Inciso 1', 'El examen de grado será público y solemne...', '8f90a1b2c3d4e5f60718293a4b5c6d7e8f90a1b2c3d4e5f60718293a4b5c6d7e'),
(16, 3, 7, 1, 'Capítulo III', 'Artículo 14', 'Inciso 1', 'El alumno debe tener promedio superior a 70...', '90a1b2c3d4e5f60718293a4b5c6d7e8f90a1b2c3d4e5f60718293a4b5c6d7e8f'),
(17, 3, 5, 2, 'Título II', 'Artículo 9', 'Inciso 2', 'La práctica requiere 320 horas cronológicas...', 'a0b1c2d3e4f5061728394a5b6c7d8e9f0a1b2c3d4e5f60718293a4b5c6d7e8f9'),
(18, 3, 11, 1, 'Título IV', 'Artículo 30', 'Inciso 1', 'Las patentes generadas pertenecen a la Universidad', 'b1c2d3e4f5061728394a5b6c7d8e9f0a1b2c3d4e5f60718293a4b5c6d7e8f9a0'),
(19, 3, 9, 1, 'Capítulo II', 'Artículo 15', 'Inciso 1', 'Es obligatorio el uso de EPP en laboratorios...', 'c2d3e4f5061728394a5b6c7d8e9f0a1b2c3d4e5f60718293a4b5c6d7e8f9a0b1'),
(20, 3, 3, 1, 'Título I', 'Artículo 6', 'Inciso 1', 'El fondo semilla otorga hasta 5 millones CLP...', 'd3e4f5061728394a5b6c7d8e9f0a1b2c3d4e5f60718293a4b5c6d7e8f9a0b1c2'),
(21, 3, 4, 1, 'Título II', 'Artículo 8', 'Inciso 1', 'El ayudante debe haber aprobado la asignatura...', 'e4f5061728394a5b6c7d8e9f0a1b2c3d4e5f60718293a4b5c6d7e8f9a0b1c2d3'),
(22, 3, 14, 1, 'Capítulo V', 'Artículo 45', 'Inciso 1', 'El acoso será sancionado con expulsión inmediata..', 'f5061728394a5b6c7d8e9f0a1b2c3d4e5f60718293a4b5c6d7e8f9a0b1c2d3e4'),
(23, 3, 6, 1, 'Título III', 'Artículo 12', 'Inciso 2', 'La cotutela exige estadía mínima de 6 meses fuera.', '061728394a5b6c7d8e9f0a1b2c3d4e5f60718293a4b5c6d7e8f9a0b1c2d3e4f5'),

```

(24, 3, 2, 1, 'Título I', 'Artículo 3', 'Inciso 1', 'Se asignan fondos para compra de licencias
Zoom...', '1728394a5b6c7d8e9f0a1b2c3d4e5f60718293a4b5c6d7e8f9a0b1c2d3e4f506'),
(25, 3, 1, 1, 'Título I', 'Artículo 1', 'Inciso 1', 'DERÓGASE el Estatuto de 1990 en su
totalidad...', '28394a5b6c7d8e9f0a1b2c3d4e5f60718293a4b5c6d7e8f9a0b1c2d3e4f50617');

-- =====
-- 5. TRAZABILIDAD (Mapeo de Grafo DAG y Activación de Triggers)
-- =====

INSERT INTO RelacionDocumental (idRespaldoLegal, idTipoRelacion, idDocumentoOrigen,
idDocumentoDestino, idFuenteDeteccion, detalleModificacion, confianza, verificada, fechaEfecto)
VALUES
-- Doc 2 complementa al Doc 1 (Estatutos orgánicos)
(1, 3, 2, 1, 1, 'Añade la figura de Vicerrectorías al estatuto.', 1.00, TRUE, '1995-06-01'),
-- Doc 4 modifica al Doc 3 (Licitaciones financieras)
(2, 1, 4, 3, 1, 'Baja el monto exigido para licitaciones a 500 UF.', 1.00, TRUE, '2005-09-01'),
-- Doc 10 complementa al Doc 6 (Reglamento Inclusión a Pregrado)
(3, 3, 10, 6, 2, 'Agrega vías de acceso STEM equitativas.', 0.85, FALSE, '2020-01-01'), -- No
verificada aún (Regex automático)
-- Doc 25 DEROGA al Doc 1 (Estatuto Orgánico Original) -> ESTO ACTIVARÁ EL TRIGGER QUE CAMBIA EL
ESTADO DEL DOC 1 A "DEROGADO"
(5, 2, 25, 1, 1, 'Derogación total del cuerpo legal de 1990.', 1.00, TRUE, '2024-01-01');

-- =====
-- 6. TABLAS PUENTE N:M (Filtros Multicriterio Universitarios)
-- =====

-- Emisores (¿Quién emitió la ley?)
INSERT INTO AreaEmisora (idDocumento, idAreaAdministrativa) VALUES
(1, 1), (2, 1), (3, 2), (4, 2), (5, 1), (6, 1), (7, 3), (8, 2), (9, 1), (10, 3),
(11, 4), (12, 4), (13, 4), (14, 4), (15, 4), (16, 3), (17, 1), (18, 5), (19, 5),
(20, 5), (21, 1), (22, 1), (23, 4), (24, 2), (25, 1);

-- Roles Afectados (¿A quién va dirigido?)
INSERT INTO DocumentoRol (idDocumento, idRol) VALUES
(1, 4), (1, 5), (6, 1), (6, 2), (7, 1), (8, 1), (9, 1), (9, 3), (11, 1), (11, 3),
(12, 1), (18, 3), (21, 1), (22, 1), (22, 2), (22, 3), (22, 4), (22, 5), (22, 6);

-- Sedes y Campus (¿Dónde aplica?)
INSERT INTO DocumentoSedeCampus (idDocumento, idRecintoUniversitario) VALUES
(1, 1), (1, 2), (1, 3), (1, 4), (1, 5), -- Estatuto en todas las sedes
(6, 1), (6, 2), (6, 4), (6, 5), -- Pregrado casi en todas
(19, 1), (19, 2), -- Laboratorios solo en sedes principales
(25, 1), (25, 2), (25, 3), (25, 4), (25, 5);

```

```
-- Departamentos y Carreras (Especificidad)
INSERT INTO DocumentoDepartamento (idDocumento, idDepartamento) VALUES
(9, 1), (9, 2), (15, 4), (19, 1), (19, 2), (19, 3);

INSERT INTO DocumentoCarrera (idDocumento, idCarrera) VALUES
(6, 1), (6, 2), (11, 3), (15, 4);

-- Jornadas
INSERT INTO DocumentoJornada (idDocumento, idJornada) VALUES
(6, 1), (6, 2), (11, 1), (15, 1), (15, 3);

-- Niveles Académicos
INSERT INTO DocumentoNivel (idDocumento, idNivel) VALUES
(6, 1), (6, 2), (11, 3), (15, 4);

COMMIT;
```

MÓDULO 6: Extensiones

Componentes Principales: Devbox, PostgreSQL 18, Extensiones (`pgvector`, `pg_cron`, `pgaudit`, `pg_permissions`, `pg_stat_statements`)

1. Introducción y Contexto

El desarrollo de sistemas de software modernos exige entornos locales estables, predecibles y aislados. En los flujos de trabajo tradicionales, la frase *"en mi máquina sí funciona"* es un problema recurrente debido a las diferencias entre los sistemas operativos de los desarrolladores (versiones de librerías, variables de entorno compiladas de forma distinta, etc.).

El desarrollo local se gestiona mediante **Devbox** para garantizar un entorno aislado, reproducible e idéntico entre colaboradores. Para el entorno de producción, la base de datos se despliega sobre **AWS RDS (PostgreSQL 18)**, realizando el aprovisionamiento de estructura, objetos y datos iniciales mediante la exportación/importación estándar con `pg_dump`.

2. Devbox

Devbox es una herramienta de línea de comandos que permite crear entornos de desarrollo aislados y reproducibles utilizando el motor de **Nix**, pero sin la compleja curva de aprendizaje de escribir expresiones de Nix puro.

Las razones fundamentales para elegir Devbox sobre alternativas tradicionales (como Docker o gestores de paquetes locales como APT/Pacman) son:

- **Aislamiento sin Sobrecarga de Virtualización:** A diferencia de Docker, Devbox no levanta contenedores ni virtualiza un kernel completo. Los paquetes se ejecutan de forma nativa en el sistema operativo del host, lo que optimiza el consumo de CPU y memoria RAM.
- **Portabilidad Absoluta:** Toda la configuración del entorno vive en un único archivo declarativo (`devbox.json`). Si otro desarrollador clona el repositorio, solo necesita ejecutar `devbox shell` para tener exactamente las mismas versiones de software compiladas de la misma manera.
- **Gestión Atómica de Paquetes:** Al estar basado en Nixpkgs, los paquetes se instalan en un store global aislado. Esto evita conflictos de versiones globales en el sistema operativo principal (por ejemplo, si la máquina host ya tiene instalado Postgres 16, no interferirá con el Postgres 18 del proyecto).

3. Arquitectura del Entorno y Paquetes Seleccionados

La configuración se definió en el archivo `devbox.json`. Tras evaluar la madurez de los paquetes del canal de Nix, se determinó el uso de las siguientes herramientas:

- **PostgreSQL 18:** Motor de base de datos relacional elegido por sus mejoras de rendimiento, concurrencia y optimización en consultas complejas.
- **postgresql18Packages.pgvector:** Extensión crítica para el almacenamiento y procesamiento de embeddings de vectores. Esencial para dar soporte a operaciones de inteligencia artificial y búsquedas de similitud en el proyecto.
- **postgresql18Packages.pg_cron:** Agendador de tareas que corre dentro de la misma base de datos, permitiendo ejecutar sentencias SQL de forma periódica sin depender de crontabs externos del sistema operativo.
- **postgresql18Packages.pgaudit:** Módulo de auditoría diseñado para registrar de manera estructurada los accesos y modificaciones a las tablas, garantizando estándares de seguridad y trazabilidad.
- **postgresql18Packages.pg_stat_statements:** Herramienta fundamental para el monitoreo del rendimiento, que permite registrar y analizar estadísticas detalladas de todas las sentencias SQL ejecutadas para identificar cuellos de botella.
- **pg_permissions** (*Instalado vía script SQL/curl*): Herramienta de auditoría de seguridad inyectada a nivel de base de datos, orientada a listar, revisar y validar de manera simple los privilegios asignados a los diferentes usuarios.

Nota de Diseño: Inicialmente se evaluó la integración de `pg_jsonschema` para la validación de esquemas JSON en base de datos. Sin embargo, debido a que PostgreSQL 18 es un lanzamiento reciente, dicha extensión aún no se encuentra empaquetada ni disponible en el canal estable de Nixpkgs para esta versión mayor de Postgres. Con el fin de no comprometer el uso de Postgres 18 y evitar "downgrades" innecesarios, se decidió remover esta dependencia y manejar las validaciones de esquemas JSON directamente en la capa de la aplicación (backend).

4. Proceso de Instalación y Configuración

El despliegue del entorno se realiza siguiendo una secuencia estricta en dos fases: el aprovisionamiento del entorno por Devbox y la posterior inicialización de bajo nivel dentro del motor de base de datos.

Fase 1: Inicialización del Entorno de Desarrollo

Para construir el entorno aislado y descargar los binarios nativos del almacén de Nix, se ejecutan los siguientes comandos en la raíz del proyecto:

```
# Actualiza las referencias de paquetes declarados en el JSON
$: devbox update

# Descarga, compila e ingresa al shell aislado con Postgres 18 y sus librerías
$: devbox shell
```

Fase 2: Configuración del Servidor

Extensiones profundas como `pg_cron`, `pgaudit` y `pg_stat_statements` modifican el comportamiento interno del motor de bases de datos. Por ende, deben ser precargadas en la memoria compartida del servidor en su arranque.

Desarrollo:

Dentro del directorio de datos generado por Postgres en el entorno virtual (`.devbox/virtenv/postgresql/data/postgresql.conf`), se añadieron las siguientes directivas de configuración:

```
# Precarga de las librerías compartidas en memoria
shared_preload_libraries = 'pg_cron, pgaudit, pg_stat_statements'

# Especificación de la base de datos objetivo para el demonio de pg_cron
cron.database_name = 'giru_db'
```

Posterior a esta edición, se procedió a reiniciar el servicio de PostgreSQL para aplicar los cambios estructurales.

Producción:

La habilitación de librerías compartidas y parámetros del motor no se realiza editando archivos físicos, sino aplicando un **DB Parameter Group** personalizado desde la consola/CLI de AWS RDS.

Fase 3: Activación en el Motor de Base de Datos

Con las librerías cargadas en el ciclo de vida del servidor, se accedió a la consola interactiva de Postgres apuntando a la base de datos del proyecto (`giru_db`) para registrar formalmente los catálogos de las extensiones:

Script SQL:

```
-- Registro y activación de la extensión de vectores
CREATE EXTENSION pgvector;

-- Registro del planificador de tareas interno
CREATE EXTENSION pg_cron;

-- Registro del módulo de auditoría de seguridad

CREATE EXTENSION pgaudit;

-- Activa el recolector de estadísticas de queries
CREATE EXTENSION pg_stat_statements;

-- Activa las funciones criptográficas (hashing, encriptación, etc.) CREATE
EXTENSION pgcrypto;
```

Fase 5: Instalación y configuración de `pg_permissions`

Como `pg_permissions` no está empaquetada en `Nixpkgs`, no la podemos añadir al archivo `devbox.json`. Sin embargo, esta extensión está construida **con SQL nativo y PL/pgSQL**. Esto significa que podemos instalarla inyectando directamente sus scripts de catálogo en el motor.

Comando ejecutado en la terminal de devbox shell:

```
curl -sL
"https://raw.githubusercontent.com/cybertec-postgresql/pg_permissions/master/pg_permissions-
-1.1.sql" | sed 's/\\echo.*//g' | psql -d giru_db
```

- **curl -sL**: Descargó el script de base de datos oficial directamente desde el repositorio de Cybertec.
- **sed 's/\\echo.*//g'**: Eliminó la línea de advertencia inicial del archivo. Esto evitó que Postgres abortara el proceso con el mensaje "Use CREATE EXTENSION...".
- **psql -d giru_db**: Inyectó secuencialmente las consultas SQL dentro de tu base de datos local.

Resultado de la Ejecución

El script creó exitosamente todos los objetos en tu esquema **public**:

- Tipos de datos personalizados (**CREATE TYPE**)
- Funciones de análisis y disparadores (**CREATE FUNCTION, CREATE TRIGGER**)
- El set completo de vistas de auditoría de seguridad (**CREATE VIEW**)

(Nota: El error final sobre **pg_extension_config_dump()** se ignoró de forma segura, ya que esa función solo se requiere cuando la extensión se registra a nivel de sistema, lo cual no afecta su uso operativo).

5. Control de Calidad y Verificación

Para asegurar el correcto aprovisionamiento de la base de datos, se debe ejecutar el comando de inspección de extensiones instaladas:

```
giru_db=> \dx
```

MÓDULO 7: Optimización y mejoras

1. Sincronización Nativa y Garantía del Índice Léxico en el Motor

1.1 Diagnóstico de Riesgos Operativos en la Gestión desde la Capa de Aplicación

Delegar la transformación del texto plano a representaciones léxicas (`tsvector`) a la capa de aplicación o servicios de backend (mediante ORMs, scripts de ingesta en Python u otros clientes) introduce vulnerabilidades arquitectónicas que comprometen la confiabilidad del sistema:

- **Riesgo de Desincronización Silenciosa (*Data Stale*):** Si ocurre un fallo en la lógica del backend, una actualización parcial no contemplada en la API, o un error en la ejecución del pipeline asíncrono, se corre el riesgo de insertar o actualizar un fragmento de texto sin recalcularlo su lexema correspondiente. Para el motor de búsqueda híbrido del sistema RAG, esta inconsistencia provoca una "pérdida silenciosa de información": el documento permanece almacenado físicamente en la base de datos, pero se vuelve invisible ante consultas léxicas por palabras clave.
- **Vulnerabilidad ante Bypasses por Ingesta Multicanal:** En un entorno de producción institucional, la base de datos no es operada exclusivamente por un único servicio. Ante la ejecución de scripts de migración masiva por parte del DBA, mantenimiento vía consola interactiva (`psql`), o la integración de futuros microservicios, cada canal requeriría reimplementar explícitamente la lógica de transformación lingüística. La omisión de este paso en un solo punto de entrada corrompe la integridad del índice global.
- **Acoplamiento Arquitectónico y Duplicación de Lógica:** La capa de aplicación no debe asumir el conocimiento de las configuraciones internas del analizador sintáctico de PostgreSQL (tales como la selección de diccionarios en español, la eliminación de *stopwords* o las reglas de lematización/*stemming*). Transferir esta responsabilidad al backend fuerza un acoplamiento innecesario entre el código de negocio y las especificidades internas del motor de datos.

1.2 Fundamentación Técnica de la Solución Nativa en PostgreSQL

Al convertir la generación del índice léxico en un atributo derivado y almacenado de manera nativa por PostgreSQL, el motor de base de datos asume el control estricto del ciclo de vida del dato, ofreciendo las siguientes garantías:

- **Atomicidad Transaccional e Integridad ACID:** La conversión del texto en lexemas se ejecuta dentro del mismo bloque transaccional que la escritura del fragmento de texto. Este mecanismo imposibilita la existencia de registros con texto plano cuyo índice léxico esté nulo o desfasado. Al confirmarse la transacción (*COMMIT*), el índice se genera de manera instantánea a nivel de motor en código C compilado nativo, asegurando una consistencia del 100%.
- **Estandarización y Centralización del Diccionario Lingüístico:** La regla de procesamiento del lenguaje natural (incluyendo el uso del diccionario en español) se centraliza en la definición del esquema de la base de datos. Esto garantiza que la totalidad de los fragmentos almacenados sean procesados bajo los mismos criterios de normalización lingüística (eliminación de tildes, conversión a minúsculas, derivación de raíces morfológicas y filtrado de conectores).
- **Descarga de Cómputo (*Offloading*) y Eficiencia Operativa:** El proceso de tokenización y análisis léxico consume ciclos de CPU. Ejecutar esta operación nativamente dentro del servidor de PostgreSQL durante la fase de almacenamiento libera a la capa de aplicación de realizar transformaciones de texto pesadas en memoria previa al envío de la consulta SQL, reduciendo la latencia general en los procesos de ingesta.
- **Fiabilidad en el Pipeline de Búsqueda Híbrida RAG:** El motor de recuperación de información para la Inteligencia Artificial puede operar bajo la premisa de que la columna léxica es una proyección exacta y actualizada del texto fragmentado. Esto asegura que los algoritmos de pre filtrado por palabras clave e intersección con embeddings vectoriales actúen sobre una fuente de verdad (SSOT) completamente libre de inconsistencias operativas.

(Nota: En el DDL oficial del Módulo 5 esta columna se crea nativamente en el *CREATE TABLE*. El siguiente script se incluye únicamente como referencia para entornos existentes que requieran una migración sin recrear la tabla).

```
-- Script de Migración (parche) en caso de aplicar sobre una base de datos ya existente:
ALTER TABLE DocumentoChunk DROP COLUMN IF EXISTS indiceLexico;
ALTER TABLE DocumentoChunk
  ADD COLUMN indiceLexico tsvector
  GENERATED ALWAYS AS (to_tsvector('spanish',
                                     coalesce(textoFragmento, ''))) STORED;
CREATE INDEX idx_chunk_lexico ON DocumentoChunk USING GIN (indiceLexico);
```

2. Control de Acceso basado en roles

2.1 Justificación Técnica y Principio de Menor Privilegio (POLP)

El diseño de la base de datos del Proyecto G.I.R.U. incorpora una arquitectura de seguridad operacional basada en **RBAC (*Role-Based Access Control*)**. Su propósito es dar cumplimiento estricto al **Principio de Menor Privilegio (*Principle of Least Privilege - POLP*)**, garantizando que los usuarios y servicios conectados únicamente posean los permisos indispensables para la ejecución de sus funciones.

En entornos de producción, la capa de aplicación (API backend en FastAPI/Flask) **nunca debe conectarse como el propietario de la base de datos ni como superusuario**. La segregación de roles previene vulnerabilidades críticas:

- **Mitigación de Inyecciones SQL:** Si el servicio web sufre una vulnerabilidad de inyección SQL, el atacante se verá restringido por los permisos del rol de la API, impidiendo la ejecución de sentencias destructivas (**DROP TABLE**, **TRUNCATE**) o la alteración de la estructura del esquema.
- **Aislamiento de Entornos DDL y DML:** Se separa drásticamente la capacidad de modificar el esquema (reservada exclusivamente para el DBA durante migraciones) de la capacidad de manipular los datos en tiempo de ejecución.
- **Trazabilidad y No Repudio:** Permite a extensiones de seguridad como **pgaudit** e **pg_permissions** asociar cada consulta ejecutada en el motor a la identidad exacta del servicio que la originó.

2.2 Matriz de Definición de Roles del Sistema

Rol en PostgreSQL	Tipo de Acceso	Ámbito de Aplicación	Permisos Otorgados
giru_app_user	Servicio (Backend / API)	Ingesta RAG, consultas y trazabilidad en producción.	Permisos DML (SELECT , INSERT , UPDATE , DELETE) en tablas del negocio; USAGE en secuencias; EXECUTE en funciones de triggers.
giru_auditor_user	Lectura (<i>Read-Only</i>)	Auditoría de ciberseguridad, reportes e inspección.	Únicamente SELECT sobre tablas, vistas de negocio y vistas de auditoría de pg_permissions .
giruAdmin	Administrador	Mantenimiento, ejecución de DDL y migraciones.	Control total del esquema (CREATE , ALTER , DROP).

2.3. Script DDL de Fortalecimiento (*Hardening*) y Asignación de Permisos

A continuación se detalla la secuencia de comandos SQL requerida para desplegar el modelo de RBAC sobre el esquema `public`:

Fase A: Revocación de Permisos Predeterminados (`PUBLIC`)

PostgreSQL otorga accesos implícitos al pseudorol `PUBLIC`. Como medida inicial de fortalecimiento de la instancia, se revocan todos los privilegios generales

```
-- Revocación de acceso predeterminado sobre el esquema public
REVOKE ALL ON SCHEMA public FROM PUBLIC;
REVOKE ALL ON ALL TABLES IN SCHEMA public FROM PUBLIC;
REVOKE ALL ON ALL SEQUENCES IN SCHEMA public FROM PUBLIC;
REVOKE ALL ON ALL FUNCTIONS IN SCHEMA public FROM PUBLIC;
```

Fase B: Configuración del Rol para el Backend (`giru_app_user`)

Se crea y parametriza el usuario de conexión que utilizará la aplicación para interactuar con las tablas transaccionales, las tablas puente y los vectores de la IA:

```
-- 1. Creación del rol con contraseña autenticada por SCRAM-SHA-256
CREATE ROLE giru_app_user WITH LOGIN PASSWORD '<<passwd_confidencial>>';

-- 2. Permiso de lectura y tránsito sobre el esquema
GRANT USAGE ON SCHEMA public TO giru_app_user;

-- 3. Permisos de manipulación de datos (DML) en todas las tablas
GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES
    IN SCHEMA public TO giru_app_user;

-- 4. Permisos de uso sobre secuencias (Requerido para llaves primarias SERIAL)
GRANT USAGE, SELECT ON ALL SEQUENCES
    IN SCHEMA public TO giru_app_user;

-- 5. Permiso de ejecución sobre funciones y procedimientos almacenados
GRANT EXECUTE ON ALL FUNCTIONS
    IN SCHEMA public TO giru_app_user;
```


Fase C: Configuración del Rol de Auditoría (*giru_auditor_user*)

Diseñado para la revisión estricta de datos sin capacidad de alteración, facilitando el cumplimiento de la Ley 21.663 de Ciberseguridad y la norma ISO 27001:

```
-- 1. Creación del usuario de auditoría
CREATE ROLE giru_auditor_user WITH LOGIN PASSWORD '<<passwd_confidencial>>';

-- 2. Otorgamiento exclusivo de lectura
GRANT USAGE ON SCHEMA public TO giru_auditor_user;
GRANT SELECT ON ALL TABLES IN SCHEMA public TO giru_auditor_user;
```

Fase D: Automatización de Permisos Futuros (*Default Privileges*)

Garantiza que cualquier nueva tabla o secuencia creada durante futuras migraciones herede automáticamente las políticas de seguridad sin requerir intervención manual:

```
-- Automatización de permisos DML para futuras tablas del esquema public
ALTER DEFAULT PRIVILEGES IN SCHEMA public
GRANT SELECT, INSERT, UPDATE, DELETE ON TABLES TO giru_app_user;

-- Automatización de uso de secuencias futuras
ALTER DEFAULT PRIVILEGES IN SCHEMA public
GRANT USAGE, SELECT ON SEQUENCES TO giru_app_user;

-- Automatización de lectura para auditoría
ALTER DEFAULT PRIVILEGES IN SCHEMA public
GRANT SELECT ON TABLES TO giru_auditor_user;
```

Nota del DBA: Todas las contraseñas y/o credenciales de este módulo deben ser solicitadas única y exclusivamente al administrador de la base de datos.